

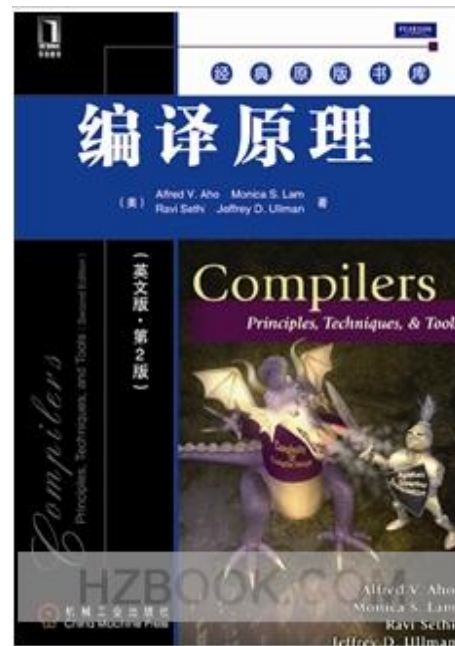


四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn

Textbook

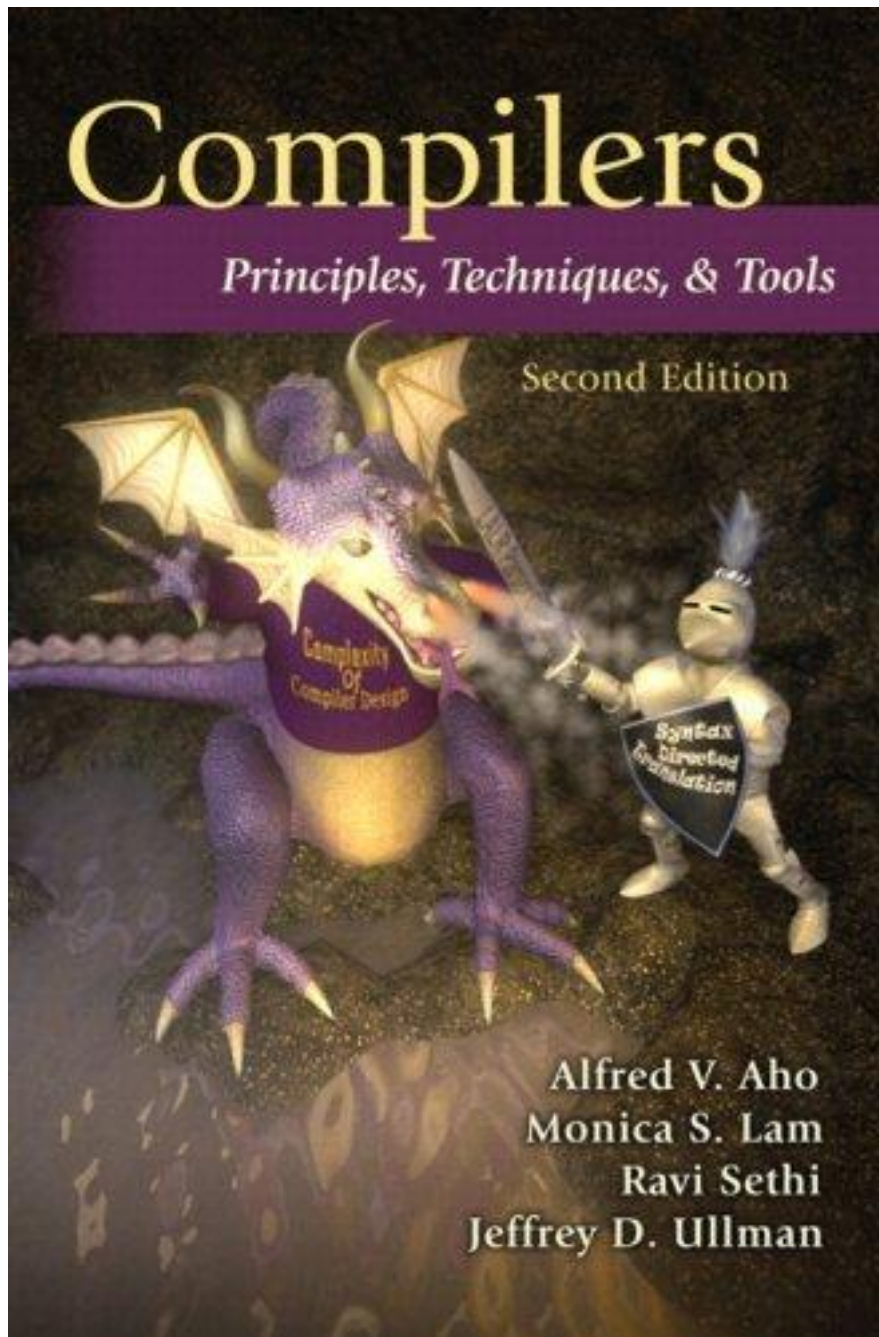


Grading

- 平时成绩 30%
- 平时测验 20%
- 期末考试 50%，强制达标分数线：50分

The Purpose of This Course

- 本课程介绍编译器构造的基本原理和基本实现技术。
- 编程实践：编译原理课程设计



Chapter 1

Introduction

1 Introduction 1

1.1	Language Processors	1
1.1.1	Exercises for Section 1.1	3
1.2	The Structure of a Compiler	4
1.2.1	Lexical Analysis	5
1.2.2	Syntax Analysis	8
1.2.3	Semantic Analysis	8
1.2.4	Intermediate Code Generation	9
1.2.5	Code Optimization	10
1.2.6	Code Generation	10
1.2.7	Symbol-Table Management	11
1.2.8	The Grouping of Phases into Passes	11
1.2.9	Compiler-Construction Tools	12

1.3	The Evolution of Programming Languages	12
1.3.1	The Move to Higher-level Languages	13
1.3.2	Impacts on Compilers	14
1.3.3	Exercises for Section 1.3	14
1.4	The Science of Building a Compiler	15
1.4.1	Modeling in Compiler Design and Implementation	15
1.4.2	The Science of Code Optimization	15
1.5	Applications of Compiler Technology	17
1.5.1	Implementation of High-Level Programming Languages	17
1.5.2	Optimizations for Computer Architectures	19
1.5.3	Design of New Computer Architectures	21
1.5.4	Program Translations	22
1.5.5	Software Productivity Tools	23
1.6	Programming Language Basics	25
1.6.1	The Static/Dynamic Distinction	25
1.6.2	Environments and States	26
1.6.3	Static Scope and Block Structure	28
1.6.4	Explicit Access Control	31
1.6.5	Dynamic Scope	31
1.6.6	Parameter Passing Mechanisms	33
1.6.7	Aliasing	35
1.6.8	Exercises for Section 1.6	35
1.7	Summary of Chapter 1	36
1.8	References for Chapter 1	38

1.1 Language Processors

Compiler

- A compiler is a program that can read **a program in one language** - the source language - and translate it into **an equivalent program in another language** - the target language.

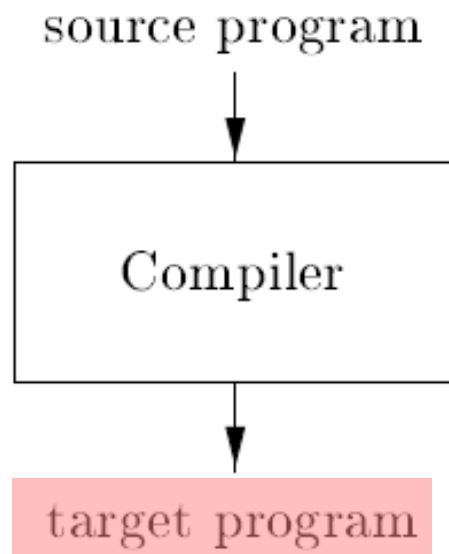


Figure 1.1: A compiler

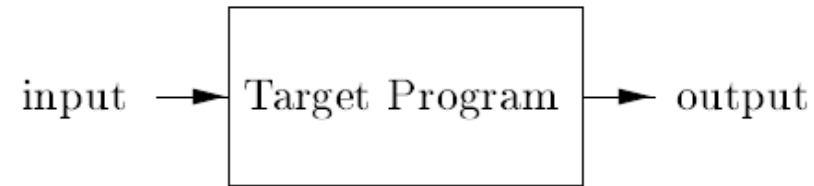
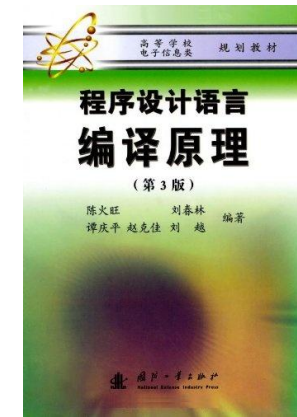
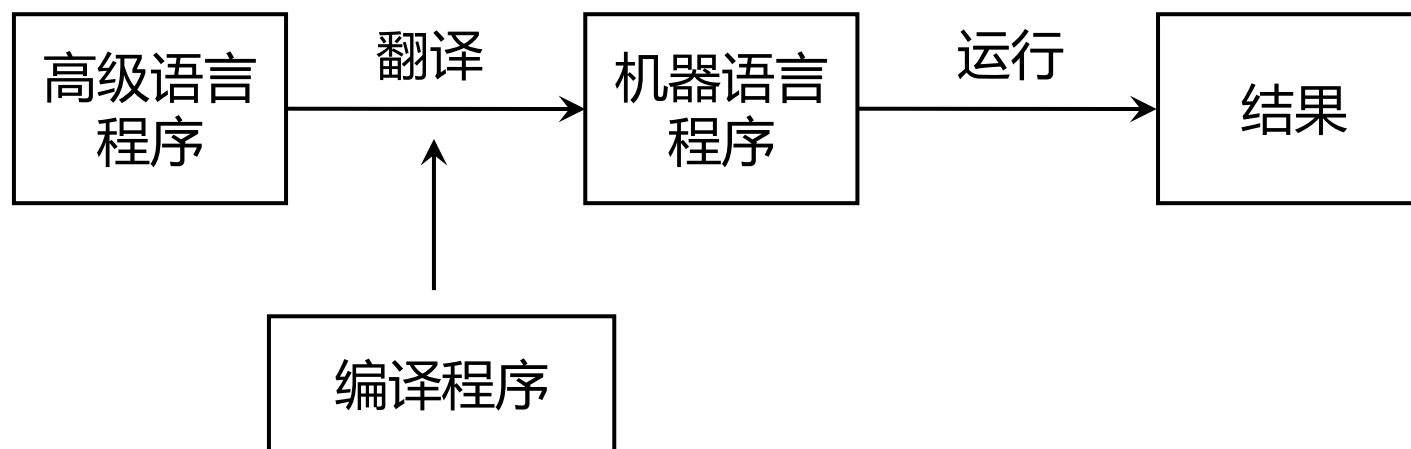


Figure 1.2: Running the target program

- 编译程序/编译器 是把某一种高级语言程序等价地转换成另一种低级语言程序（如汇编语言或机器语言程序）的程序。



Interpreter

- An interpreter is another common kind of language processor.

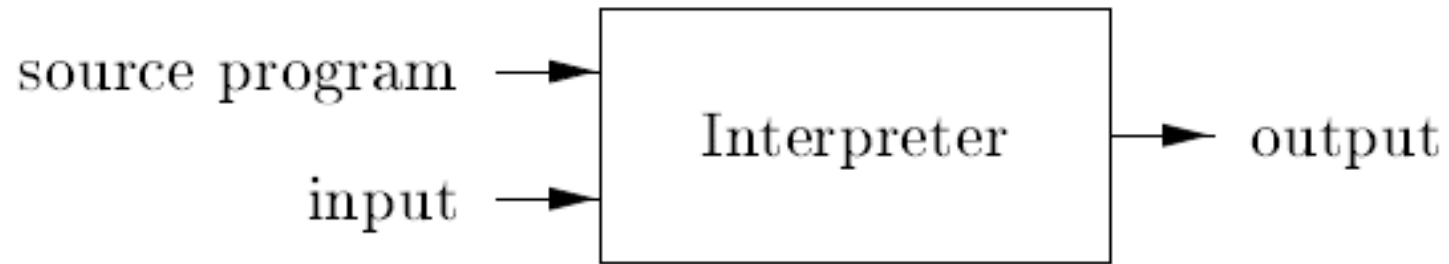


Figure 1.3: An interpreter

Compiler vs. Interpreter

source program

```
.....  
b=1;  
a=b+2;  
write a;  
.....
```

Compiler

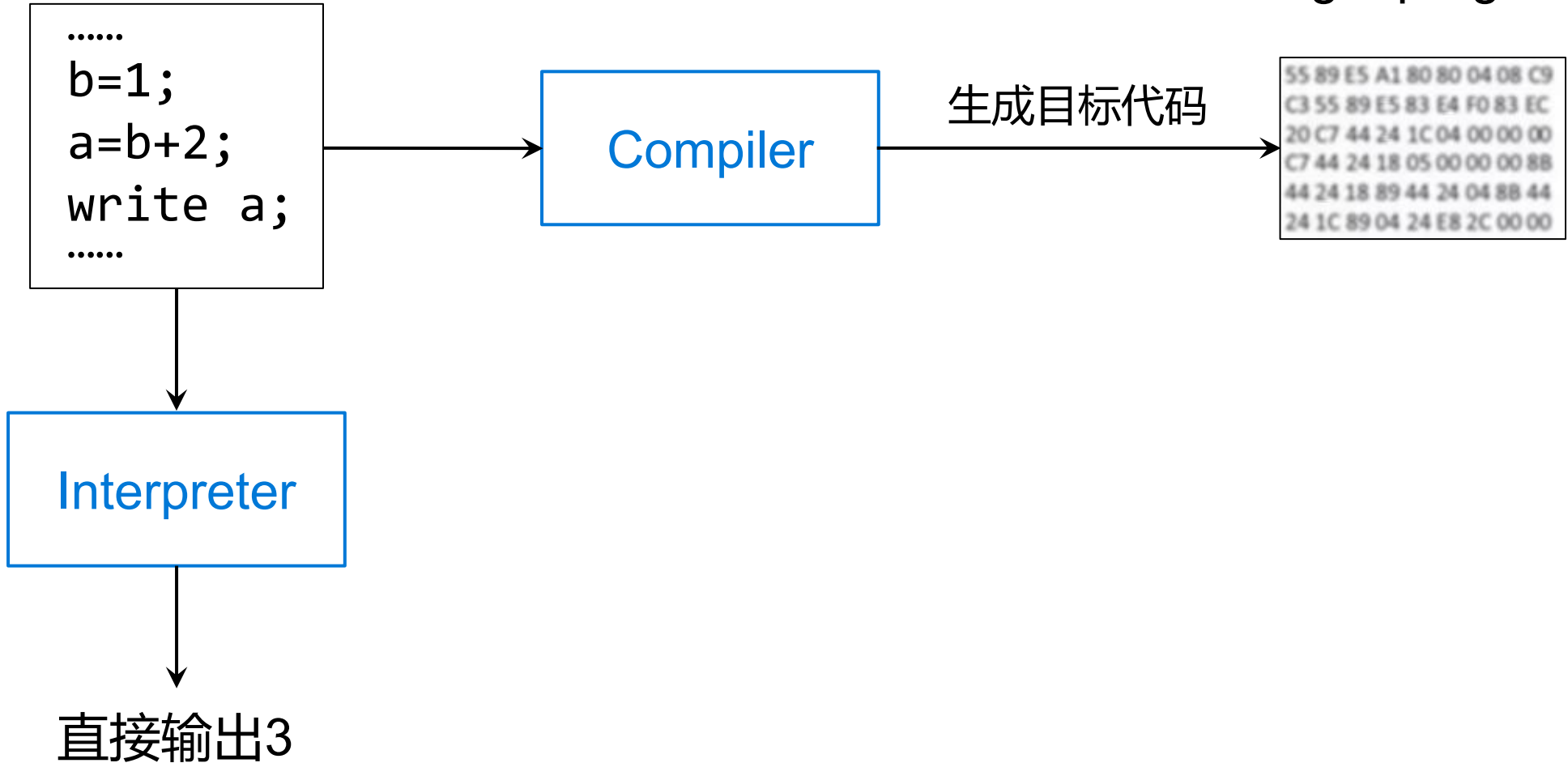
生成目标代码

target program

```
55 89 E5 A1 80 80 04 08 C9  
C3 55 89 E5 83 E4 F0 83 EC  
20 C7 44 24 1C 04 00 00 00  
C7 44 24 18 05 00 00 00 8B  
44 24 18 89 44 24 04 8B 44  
24 1C 89 04 24 E8 2C 00 00
```

Interpreter

直接输出3



Hybrid Compiler

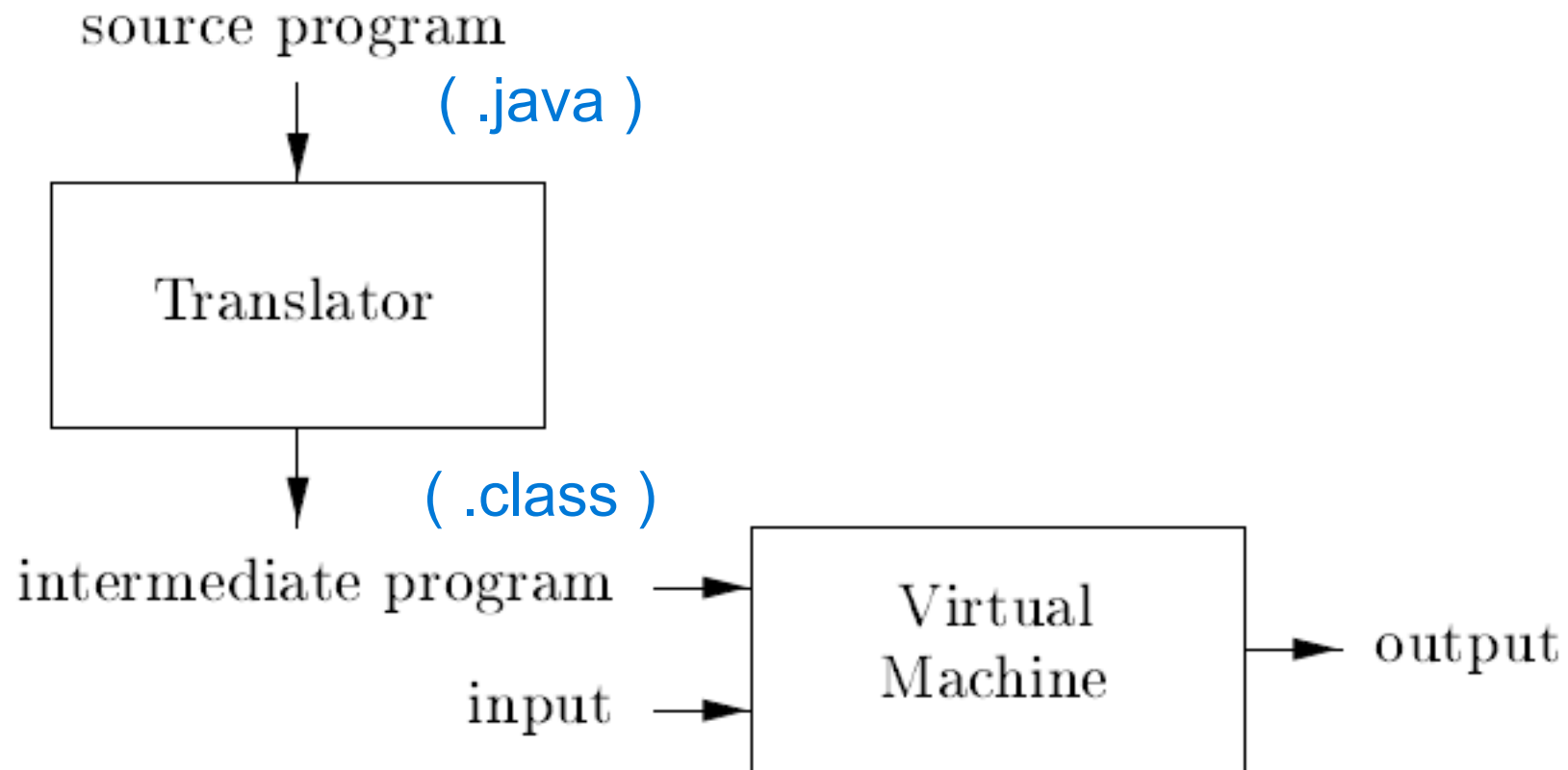
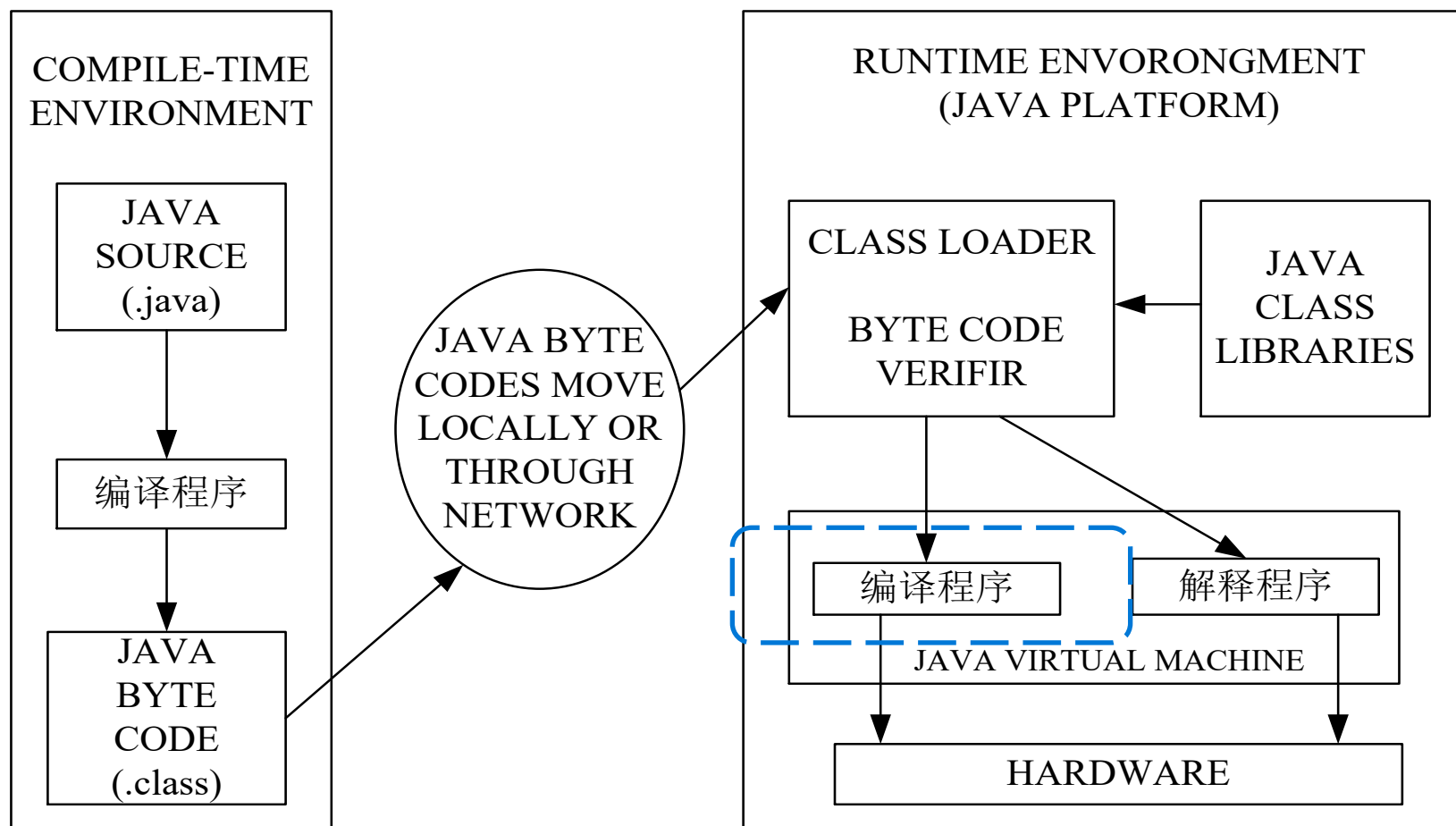


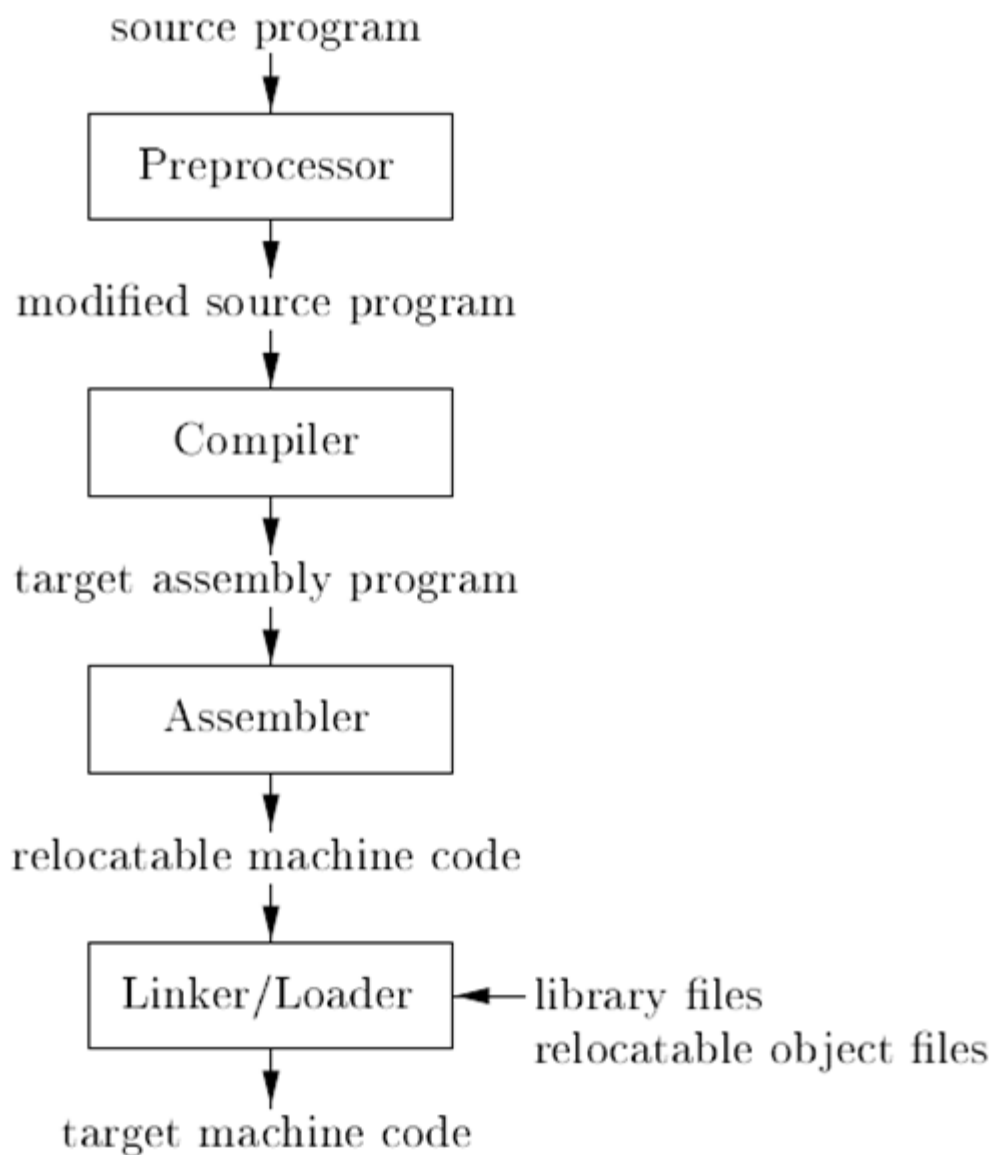
Figure 1.4: A hybrid compiler



—— 张素琴等, 《编译原理》(第2版), 清华大学出版社

- **Just – in – Time (JIT编译)**

当JVM发现某个方法或代码块运行特别频繁的时候, 会认为这是“热点代码”(Hot Spot Code)。然后JIT把“热点代码”编译成本地机器相关的机器码, 并进行优化, 再把机器码缓存起来, 以备下次使用。



- 在编译之前，有一些预处理的工作由[预处理器](#)进行，例如，把头文件合并到源程序中，把宏定义转换为源语言的语句等等。
- 经过预处理的源程序作为输入传递给[编译器](#)。编译器可能产生一个汇编语言程序作为输出，因为汇编语言相对于机器语言而言，比较容易调试。
- 汇编语言程序由[汇编器](#)进行处理，生成可重定位的机器代码。
- 大型程序通常会分解成多个部分进行编译。[链接器](#)把各个目标文件以及库文件连接到一起，形成真正在机器上运行的代码，也就是可执行程序。
- [加载器](#)把可执行程序加载到内存中执行。

Figure 1.5: A language-processing system

1.2 The Structure of a Compiler

自然语言翻译 VS. 程序设计语言翻译

A compiler is a program that can read a program in one language and translate it into an equivalent program in another language.

```
max = a >= b ? a : b;
```

lexical analysis

syntax analysis

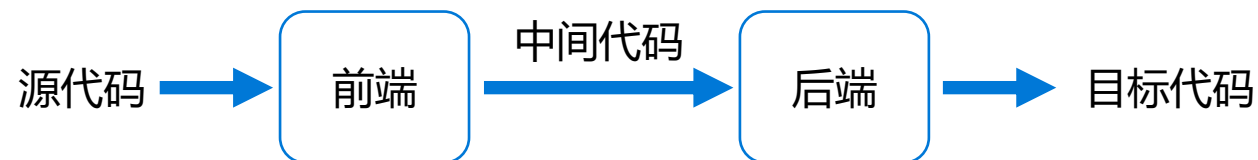
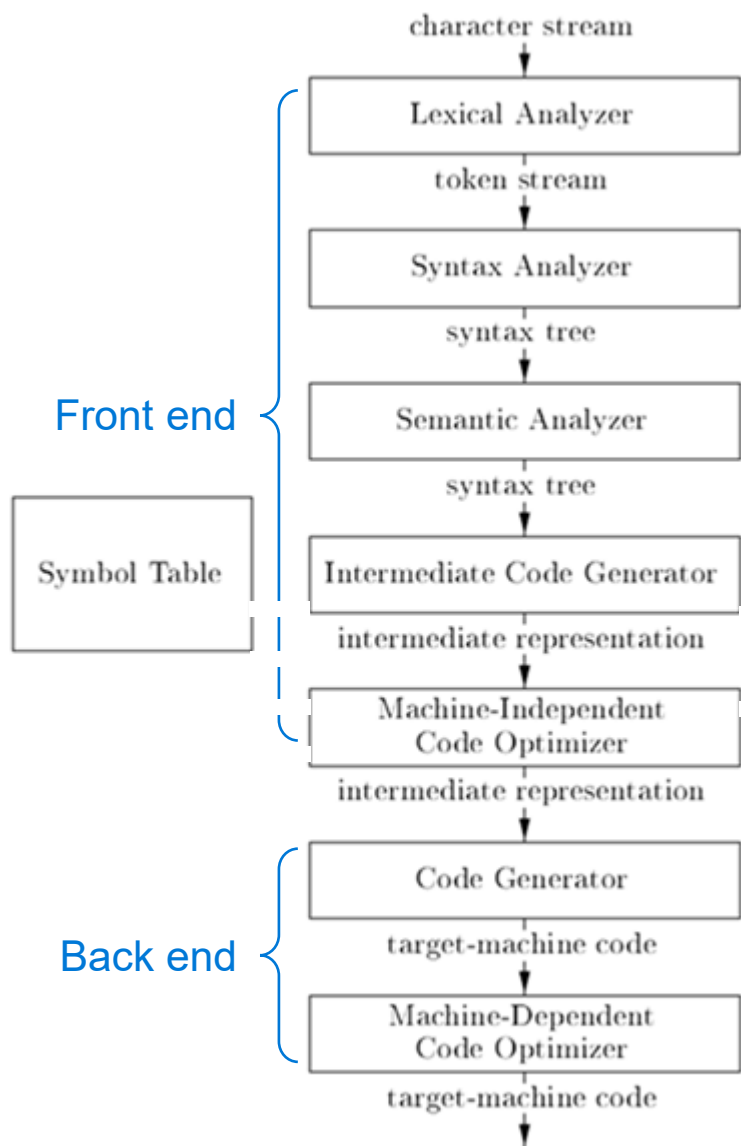
semantic analysis

intermediate code generation

optimization

target code generation

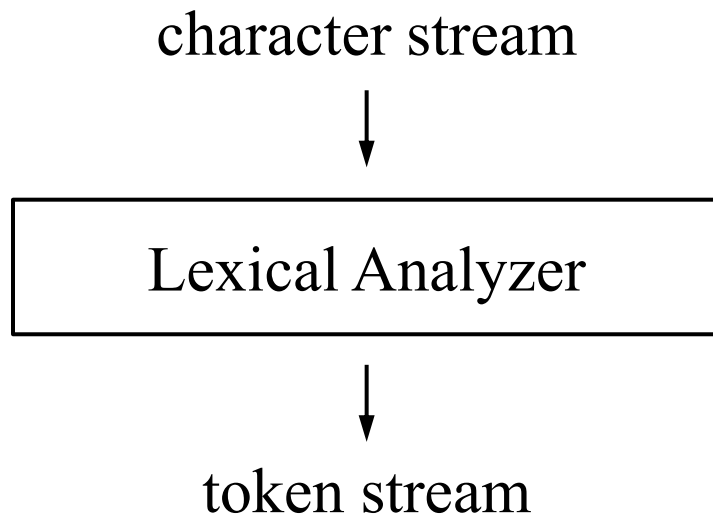
Logical Phases of a Compiler



- **前端** 与源语言有关，如词法分析，语法分析，语义分析，以及中间代码生成，与机器无关的优化
- **后端** 与目标机有关，包括与目标机有关的优化，目标代码生成
- **优点** 程序逻辑结构清晰；优化更充分，有利于移植。

Figure 1.6: Phases of a compiler

1.2.1 Lexical Analysis / Scanning



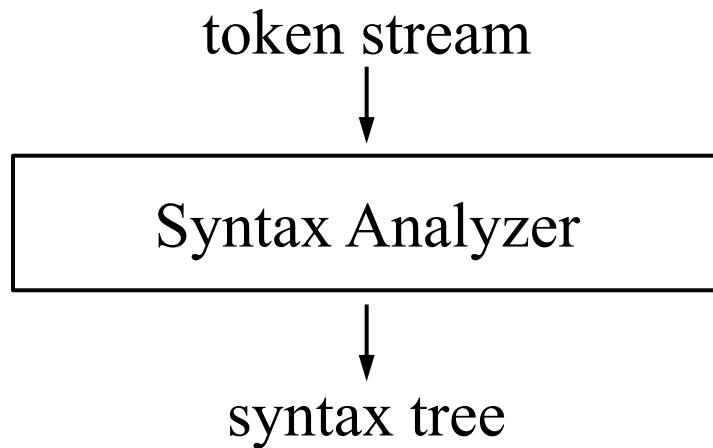
```
position = initial + rate * 60
```

Lexeme	Token	Token
position	<id, "position">	<id, 1>
=	<=>	<=>
initial	<id, "initial">	<id, 2>
+	<+>	<+>
rate	<id, "rate">	<id, 3>
*	<*>	<*>
60	<NUM, 60>	<60> or <NUM, 4>

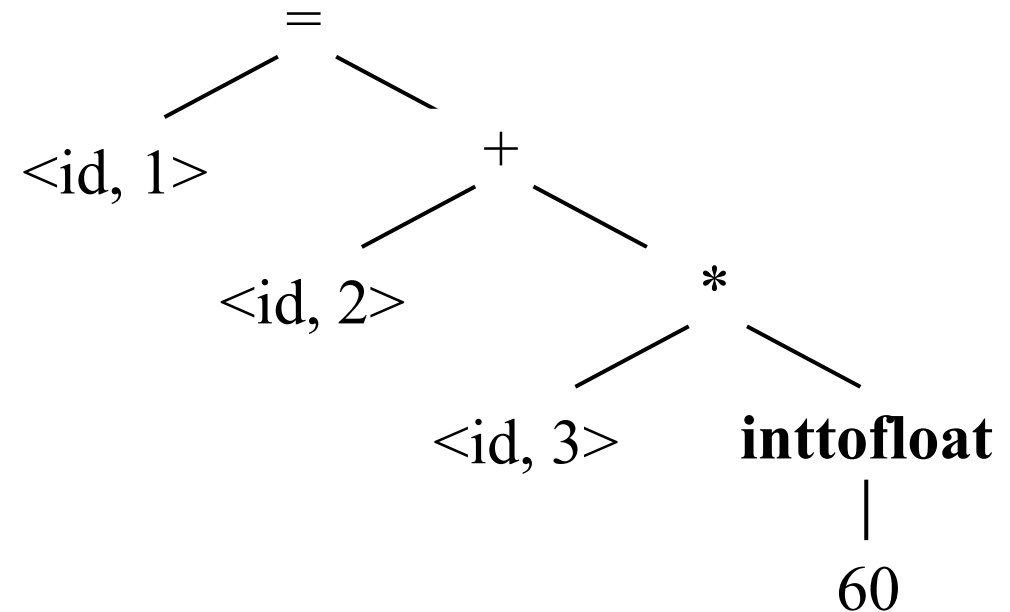
```
<id,1> <=> <id,2> <+> <id,3> <*> <60>
```

- **token** <token-name, attribute-value>
- **lexeme** the sequence of input characters that a token represents.

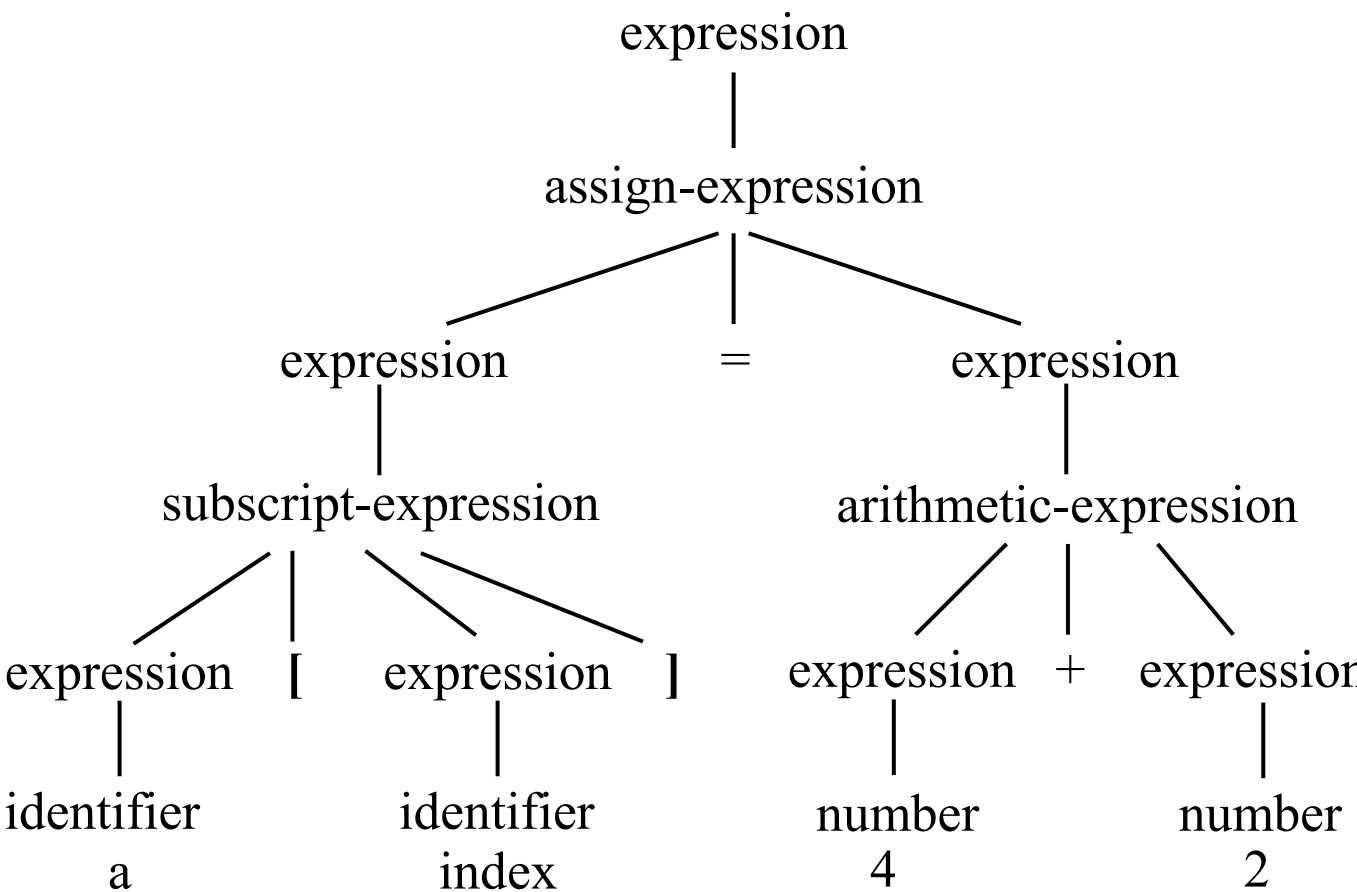
1.2.2 Syntax Analysis



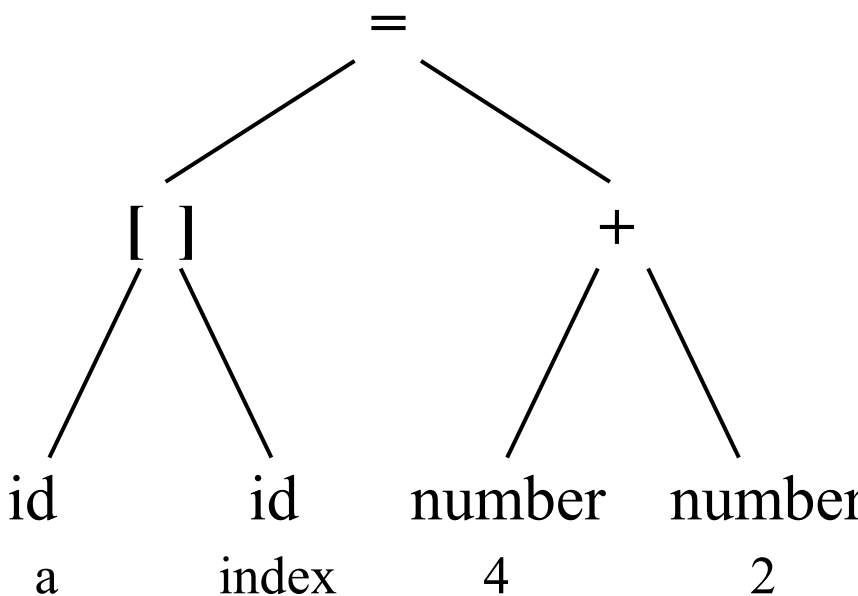
`<id,1> <=> <id,2> <+> <id,3> <*> <60>`



```
a[index] = 4 + 2
```



syntax tree

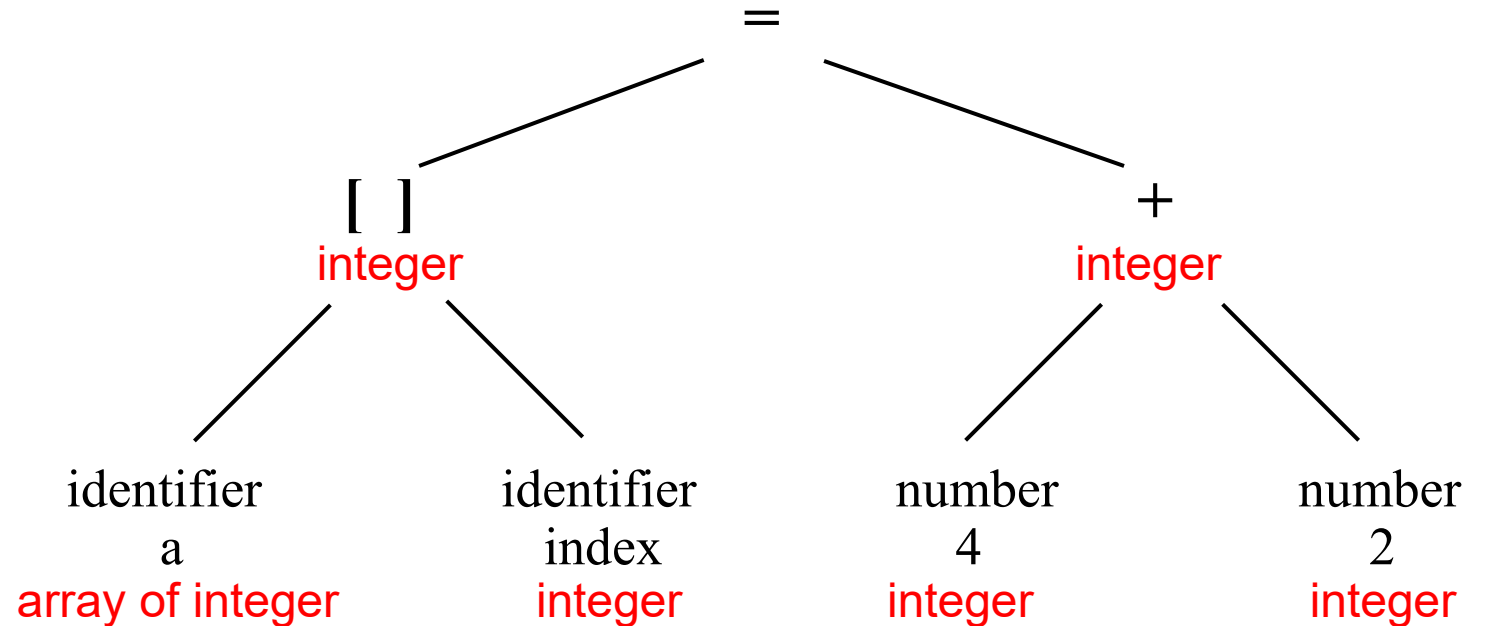
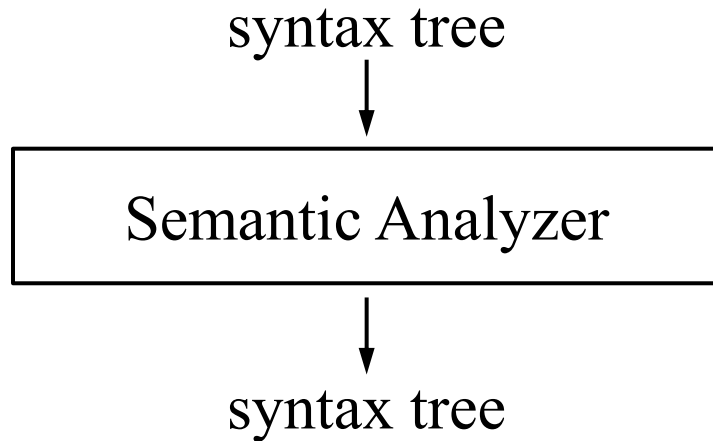


abstract syntax tree



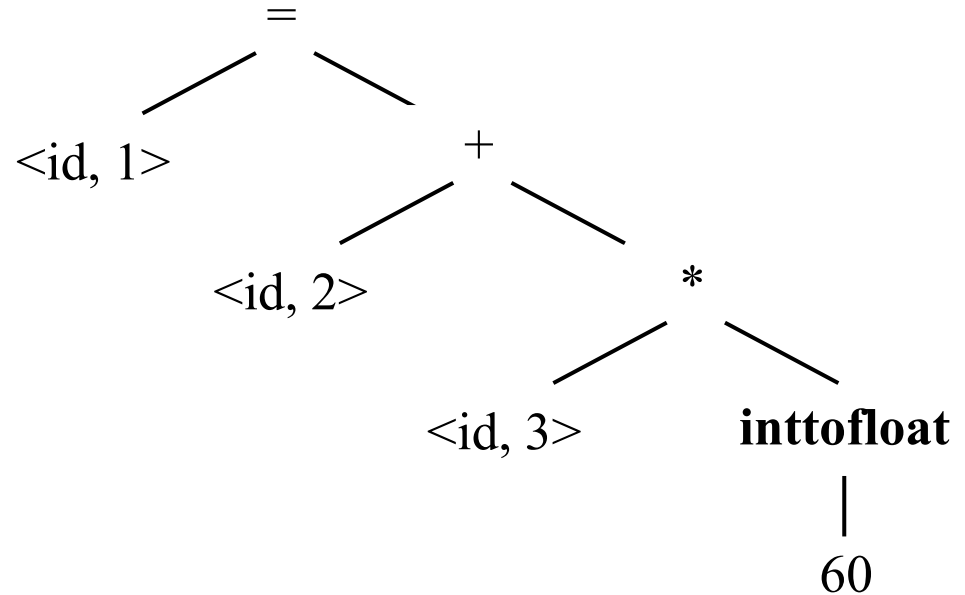
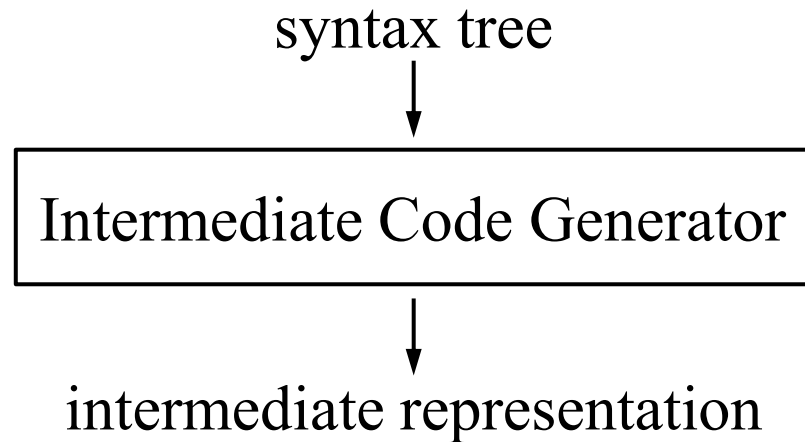
1.2.3 Semantic Analysis

a[index] = 4 + 2



annotated parse/syntax tree

1.2.4 Intermediate Code Generation

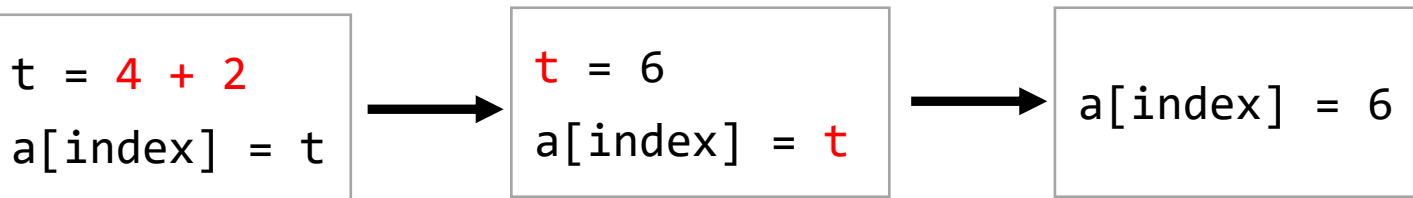
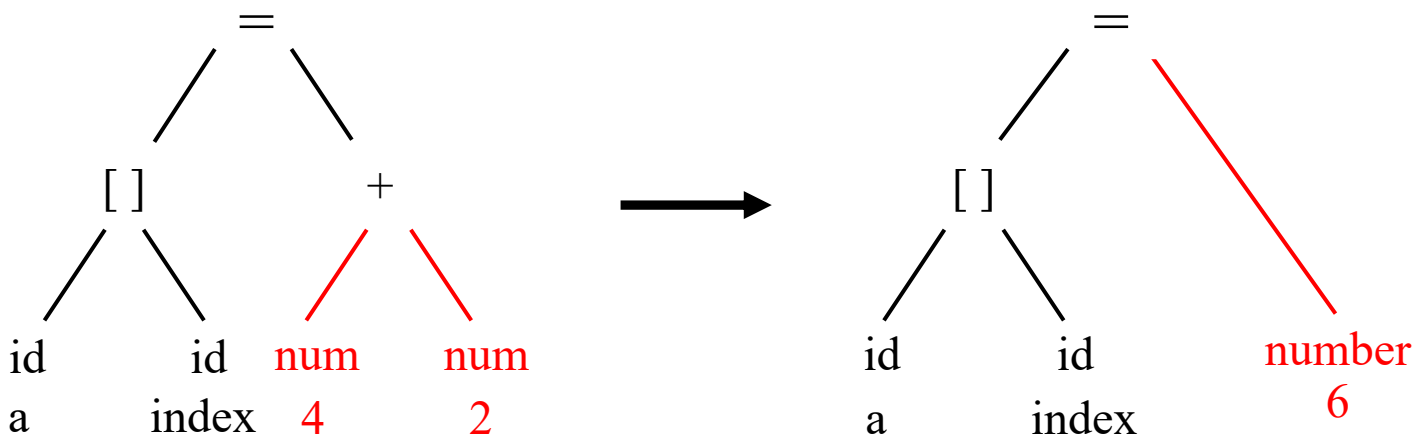


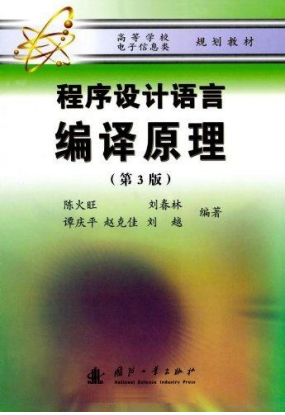
```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1= t3
```

1.2.5 Code Optimization

- 机器无关优化 / 机器相关优化
- 不同的编译器所做的代码优化工作量相差很大。
- 可以在不同的中间表示形式上进行优化。
- 常用的优化方法

- 删除公共子表达式
- 删除无用代码
- 常量合并
- 代码移动
- 强度削弱
- 删除归纳变量





```
for K:=1 to 100 do
begin
  X := I + 1;
  M := I + 10 * K;  N := J + 10 * K
end
```

400次加法
200次乘法 ➡ 301次加法

序号	OP	ARG1	ARG2	Result	注释
(1)	:=	1		K	K:=1
(2)	j<	100	K	(9)	if (100<K) goto (9)
(3)	+	I	1	X	X:=I+1
(4)	*	10	K	T1	
(5)	+	I	T1	M	
(6)	*	10	K	T2	
(7)	+	J	T2	N	
(8)	+	K	I	K	
(9)	j			(2)	goto (2)

序号	OP	ARG1	ARG2	Result	注释
(1)	+	I	1	X	X:=I+1
(2)	:=	I		M	M:=I
(3)	:=	J		N	N:=J
(4)	:=	1		K	K:=1
(5)	j<	I00	K	(9)	if (100<K) goto (9)
(6)	+	M	10	M	M:=M+10
(7)	+	N	10	N	N:=N+10
(8)	+	K	1	K	
(9)	j			(4)	goto (5)

1.2.6 Code Generation

intermediate representation



Intermediate Code Generator



target-machine code

- 依赖于硬件系统结构和机器指令集
- 目标代码三种形式
 - 绝对指令代码：可直接运行
 - 可重定位指令代码：需要连接装配
 - 汇编指令代码：需要进行汇编

x = 2



Mov x,2



C7 06 0000 0002

position = initial + rate * 60



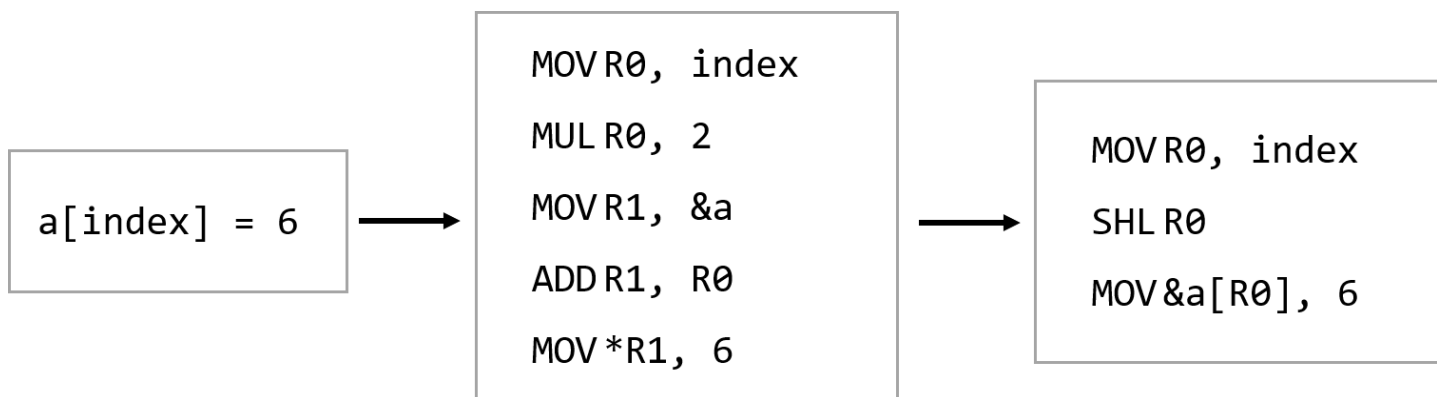
t1 = id3 * 60.0
id1 = id2 + t1



LDF R2,id3
MULF R2,R2,#60.0
LDF R1,id2
ADDF R1,R1,R2
STF id1,R1

Machine-Dependent Code Optimizer

- 寄存器的选择
- 更高效的机器指令的选择
- 更有效的编址模式



1.2.7 Symbol Table

- 符号表是编译器的重要数据结构。

- 变量，常量

- 函数

- 函数名

- 参数数量和类型

- 参数的传递方式

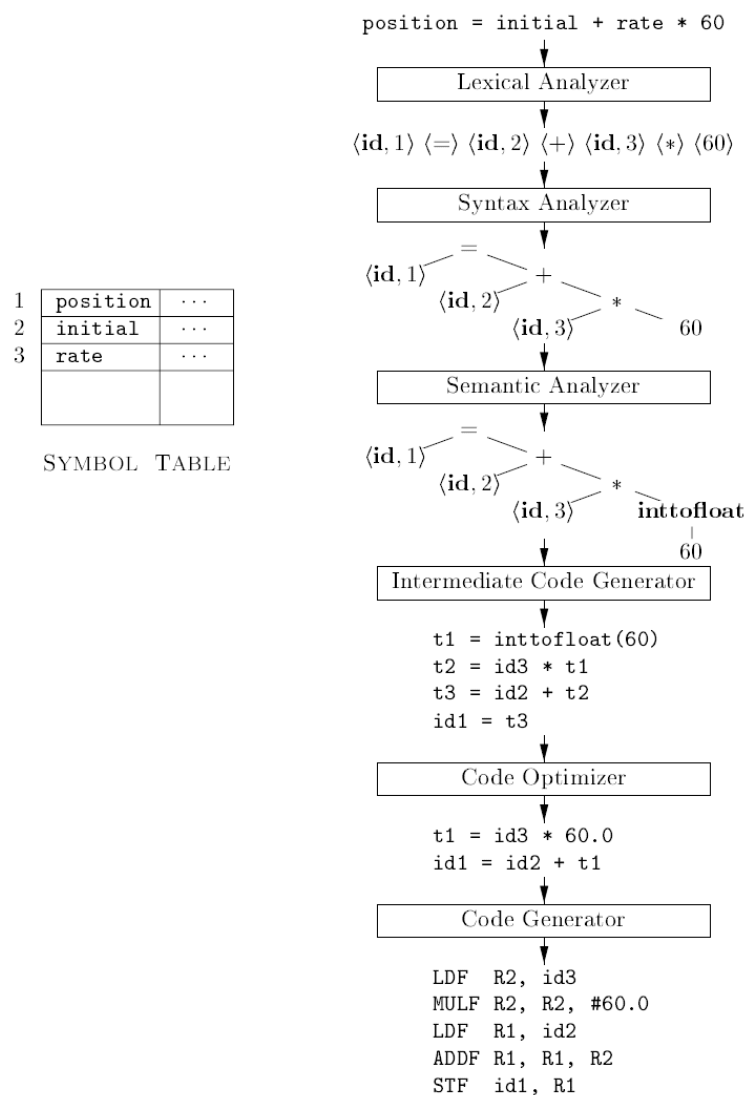
- 以及返回类型等

```
position = initial + rate * 60
```

position	类型、存储宽度、相对地址
initial	类型、存储宽度、相对地址
rate	类型、存储宽度、相对地址
.....	

- 符号表的具体实现也是一个很重要的问题。

1.2.8 Passes



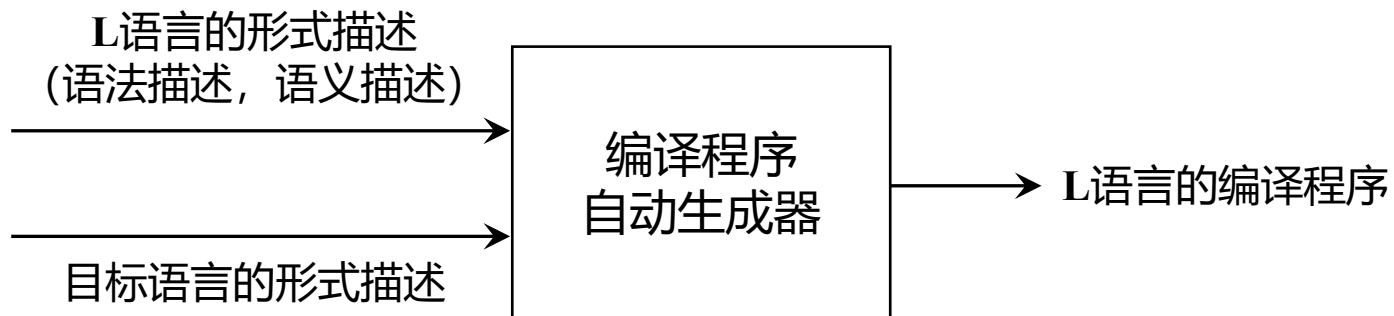
- 阶段 (phase) 与遍 (pass) 是两个不同的概念。
- 遍：对源程序或源程序的中间表示从头到尾扫描一次。
- 编译器可以是一遍，但代码效率不高。
 - PASCAL语言和C语言都允许单遍编译。
- 大多数带有优化的编译器都需要超过一遍，典型的安排是
 - 第1遍用于扫描和分析，
 - 第2遍用于语义分析和源代码层优化，
 - 第3遍用于代码生成和目标层的优化。
- 更深层的优化则可能需要更多的遍：5遍、6遍、甚至8遍都是可能的。
- TINY 编译器

```
syntaxTree = parse();
buildSyntab(syntaxTree);
typeCheck(syntaxTree);
codegen(syntaxTree, codefile);
```

Figure 1.7: Translation of an assignment statement

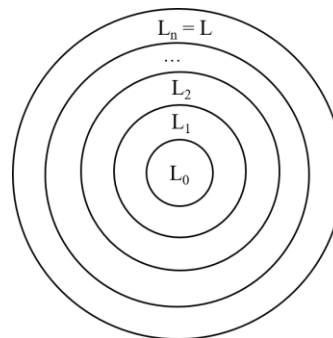
1.2.9 Compiler-Construction Tools

- 词法分析程序生成器：Lex, Flex
- 语法分析程序生成器：Yacc, Bison, ANTLR



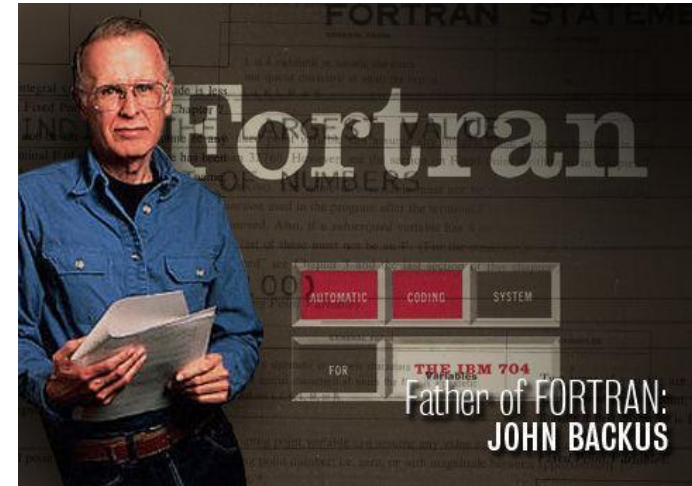
相关概念

- 编写编译器的语言：机器语言 / 汇编语言 / 高级程序设计语言
- 自编译
 - 使用某种高级语言来编写自己的编译程序。
 - 高级语言的自编译性是指，如果一个高级语言能用来书写它自己的编译程序，则该语言称为自编译语言。
- 自展
 - 用机器语言或汇编语言编写一个S语言的不完善的编译器，再用这个编译器来编译用S语言编写的S语言的编译器，多次重复之后得到一个完善的编译器。
 - 把源语言L分解成一个核心部分 L_0 与扩充部分 $L_1, L_2, \dots, L_n$ ，用机器语言或汇编语言编写 L_0 的编译程序 T_0 ，然后用 L_0 写 L_1 的编译程序 T_1 ，再用 L_1 编写 L_2 的编译程序，……，直到得到源语言L的编译程序。
- 交叉编译
 - 交叉编译是指用A机器（宿主机）上的编译程序来产生在B机器（目标机）上运行的目标代码。
 - 编译源代码的平台和执行源代码编译后程序的平台是两个不同的平台。例如，在Intel x86架构 / Linux平台下，使用交叉编译工具链生成的可执行文件，在ARM架构 / Linux下运行。



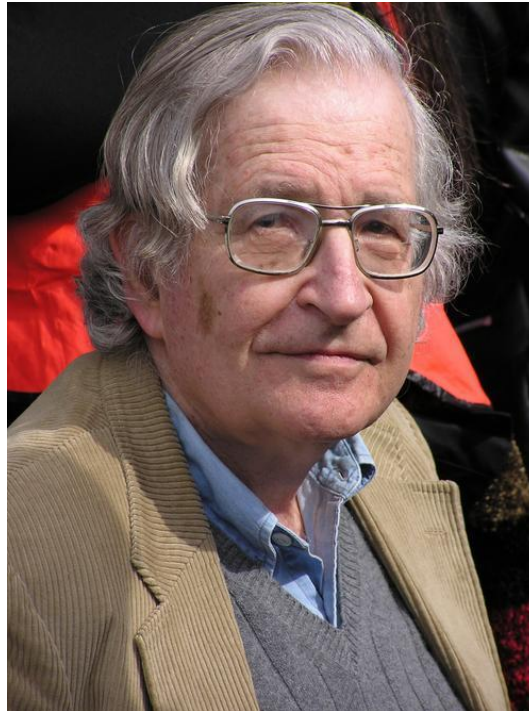
Brief History of Compiler

- The first compiler, known as A-0, was implemented by Grace Hopper and her team in the 1950s.
- The first compiler was developed by a team at IBM led by John Backus between 1954 and 1957.



Brief History of Compiler

- The structure of natural language was studied at about the same time by Noam Chomsky.



Brief History of Compiler

- The related theories and algorithms in the 1960s and 1970s
 - Chomsky hierarchy;
 - Finite automata, Regular expressions;
 - Optimization techniques or code, improvement techniques;
 - Parser generators such as Yacc by Steve Johnson in 1975 for the Unix system; Scanner generators such as Lex by Mike Lesk for Unix system about same time
- Recent advances in compiler design
 - Compilers have included the application of more sophisticated algorithm.
 - Compilers have become more and more a part of IDE.
 - Parallel compiler technology for high-performance computing
 - Cross-compiler technology for embedded computing
- However, the basic of compiler design have **not changed** much **in the last 20 years**.

Original edition copyright ©1997 by PWS Publishing Company.



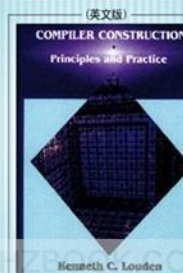
The Features of Tiny Language

- 语句序列由分号隔开;
- 无过程, 无声明。
- 所有变量都是整型变量, 通过对变量赋值即可声明变量。
- 两个控制语句: if 和 repeat。
 - 这两个控制语句本身也可包含语句序列。
 - if 语句有一个可选的else部分且必须由关键字end结束。
- read语句和write语句实现输入/输出。
- 注释包含在一对花括号中, 注释不能嵌套。
- 表达式包括两种: 布尔表达式和整型算术表达式。
 - 布尔表达式由对两个算术表达式的比较组成, 比较运算符为< 和 = 。
 - 布尔表达式只能出现在控制语句中。
 - 算术表达式可以包括整型常数、变量、圆括号以及4个整型运算符+、-、*、/。

```
{ Sample program
  in TINY language -
  computes factorial
}
read x; { input an integer }
if 0<x then { don't compute if x <= 0 }
  fact := 1;
  repeat
    fact := fact * x;
    x := x - 1
  until x = 0;
  write fact { output factorial of x }
end
```

- 通过递归函数调用实现阶乘计算?

编译原理与实践



The features of C-Minus Language

- C-Minus语言比Tiny语言复杂，比C简单。
本质上是C的一个子集。
- C-Minus包括
 - 整型变量、整型数组以及函数；
 - 局部变量、全局变量和递归函数；
 - if 语句和 while 语句，等等。
- 程序由函数序列和变量声明序列组成。
- 程序最后必须是一个main函数。

```
/* One Program in C-;  
   Input/output is implemented by  
   read() and write(). */  
  
int fact(int x)  
{  
    if (x>1)  
        return x*fact(x-1);  
    else  
        return 1;  
}  
  
void main()  
{  
    int x;  
    x = read();  
    if (x>0) write(fact(x));  
}
```



End of Chapter One